

Del Big-O al Big-Win: Diagnóstico y Optimización de Rendimiento

Computación II UNLaR



CONTENIDOS



01

Intro: Teoría vs Realidad

02

Factores Ocultos

03

Profiling: Arqueología
Digital

04

Metodología y Casos

05

Laboratorio y Aplicación

06

Errores, Futuro y Cierre



01

Intro: Teoría vs Realidad



“ ...

Portada: Cruce del puente teórico

01

Bienvenida

Bienvenidos al laboratorio donde la notación $O(n)$ se encuentra con el cronómetro. Hoy aprenderemos a ver el código no solo como algo que funciona, sino como algo que debe funcionar eficientemente en producción.

02

Ejemplos en la industria

Netflix invierte millones en performance engineering y Google emplea ingenieros para ganar 50 ms en cada búsqueda. Estos ejemplos demuestran la importancia de la optimización en el mundo real.

03

Cambio de mentalidad

Esta clase cambiará su mentalidad de desarrollador a ingeniero de performance, enseñándoles a medir, entender y mejorar el rendimiento de sus aplicaciones.

Agenda: Revelaciones y herramientas reales



01

Agenda de la clase

Hoy revelaremos por qué sus predicciones teóricas fallan y les enseñaré las herramientas que Facebook usa para optimizar el News Feed, Amazon para acelerar recomendaciones y Tesla para que el autopilot responda en tiempo real.

02

Contenido práctico

La agenda incluye mitos, factores ocultos, jerarquía de memoria, tipos de profiling, laboratorios prácticos y conexión con trabajos prácticos para que lleven el conocimiento a sus proyectos.



Pregunta disparadora: ¿Por qué $O(n)$ pierde?

...

Importancia de la pregunta

Hoy entenderán exactamente por qué sucede y, más importante, cómo detectarlo y solucionarlo antes de que los usuarios lo experimenten.

Objetivo práctico

El objetivo es que puedan identificar y resolver estos problemas antes de que afecten la experiencia del usuario, mejorando así la calidad y eficiencia de sus aplicaciones.

01

Situación común

Muchos han visto un algoritmo $O(n)$ perder contra uno $O(n \log n)$. Es frustrante y común: implementan la solución teóricamente superior y en la práctica es más lenta.

02

03

Transformación en ingenieros

Esta pregunta los transformará en ingenieros de performance capaces de explicar y corregir estas paradojas que a menudo ocurren en el desarrollo de software.

04

El mito: Big-O no es oráculo



Destrucción del mito

Destrozamos el mito de que la complejidad asintótica predice perfectamente el rendimiento. He visto startups elegir algoritmos solo por Big-O y luego sufrir lentitud. Bubble Sort, despreciado, vence a Quick Sort para arrays pequeños por menor overhead; Facebook lo usa internamente para arrays <10 elementos. Las constantes multiplicativas, localidad, overhead de recursión y optimizaciones del compilador deciden la carrera real.



“ ...

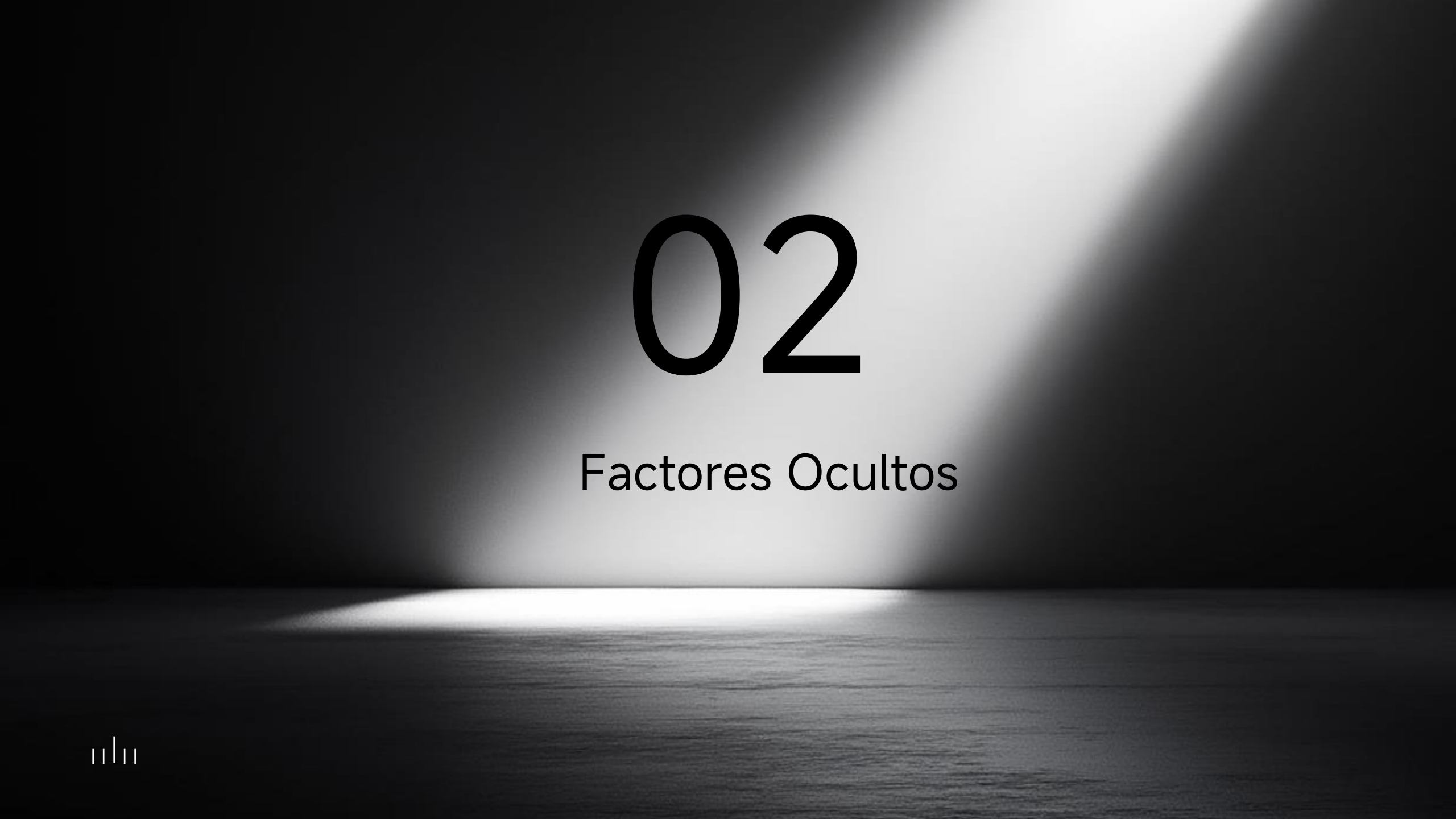
Definiciones: Predicción vs realidad

Rendimiento teórico

Rendimiento teórico es la predicción estadística según análisis matemático, como el pronóstico del clima, útil para planificar pero no garantiza sol.

Rendimiento real

Rendimiento real es el tiempo que experimenta el usuario, incluyendo temperatura del CPU, RAM disponible y carga del sistema. La brecha entre ambos es lo que hoy aprenderán a medir, entender y minimizar con herramientas profesionales.

A dramatic scene with a bright spotlight illuminating a dark stage floor. In the center of the light, the large number '02' is displayed in a bold, black, sans-serif font.

02

Factores Ocultos



Factores que impactan la ejecución real

...

01

Lista abrumadora

Lista abrumadora intencionalmente: hardware, sistema operativo, compilador, patrones de datos y entorno. Cada factor puede ser el culpable de que su $O(n)$ pierda contra $O(n^2)$.

02

Importancia de medir

Intel emplea miles de ingenieros para entender estas interacciones. Las empresas serias nunca optimizan por intuición; siempre miden.

03

Detective forense digital

Comprender esta red de influencias es el primer paso para convertirse en detective forense digital de performance, capaz de identificar y resolver problemas de rendimiento.

04

Objetivo de la sección

Esta sección les enseñará a identificar y medir estos factores ocultos, para que puedan tomar decisiones informadas y mejorar el rendimiento de sus aplicaciones.

Jerarquía de memoria: La tabla que debes saber de memoria

...

Números críticos

01

Estos números deben estar tatuados en su cerebro: acceso a registros ~1 ciclo, caché L1 ~3-4, L2 ~10-20, L3 ~40-75, RAM ~200-300, SSD ~500 000, HDD ~20 000 000.

Ejemplo práctico

02

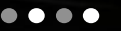
La diferencia entre L1 y RAM es como caminar a la cocina versus viajar a Buenos Aires. Muchos algoritmos teóricamente eficientes fallan por ignorar esta realidad.

Aplicación en la industria

03

Los ingenieros de videojuegos conocen estos números y por eso logran 60 FPS. Comprender la jerarquía de memoria es esencial para desarrollar software eficiente.

Localidad de caché: El factor invisible



01

Definición de localidad

Localidad espacial accede datos cercanos en memoria; localidad temporal reutiliza datos recientes. Es la diferencia entre leer un libro página por página versus saltar aleatoriamente.

02

Aplicación en bases de datos

MySQL, PostgreSQL y todas las BD modernas están diseñadas obsesivamente alrededor de este concepto. Sus índices B-tree no solo son elegantes teóricamente; minimizan caché misses para acelerar consultas reales.





Ejemplo práctico: Cache misses en acceso a matriz

Impacto de patrones de acceso

Código idéntico algorítmicamente puede ser 10× más lento. Recorrer matriz por filas carga líneas de caché secuencialmente; por columnas salta por toda la memoria causando misses constantes. NumPy y bibliotecas científicas están obsesionadas con estos patrones. Comprender y controlar el acceso a memoria es la habilidad que separa al ingeniero de performance del programador teórico.



03

Profiling: Arqueología Digital

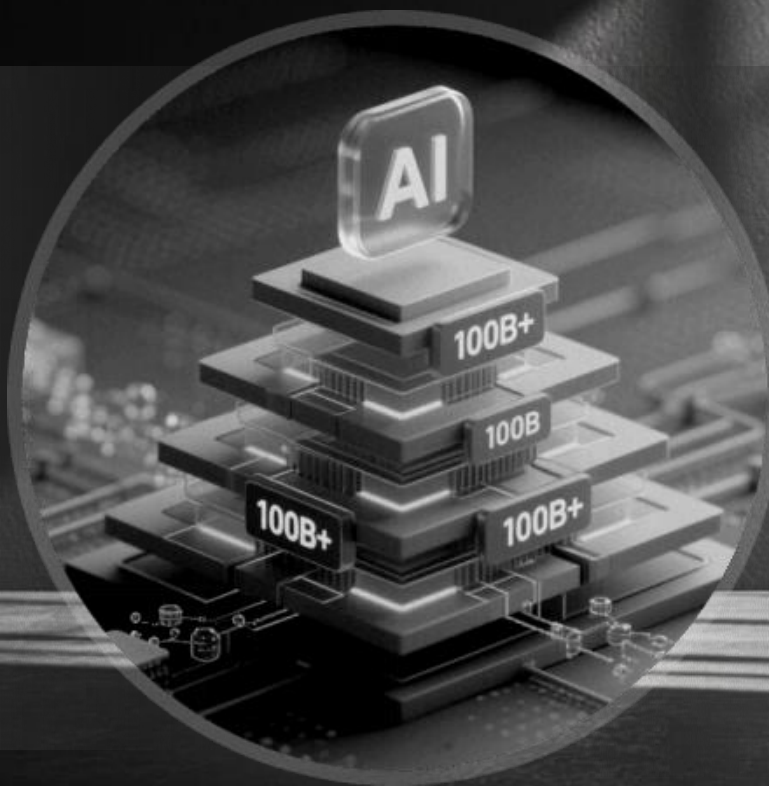


¿Qué es profiling? Arqueología del código

...

Definición de profiling

Profiling es análisis sistemático del rendimiento durante ejecución y también arqueología digital: excavar en el código para encontrar artefactos que explican su comportamiento.



Importancia de la evidencia

Es la diferencia entre saber que el auto consume mucha gasolina versus saber exactamente qué componente causa el consumo. Con profiling dejan de adivinar y empiezan a diagnosticar con evidencia, convirtiéndose en ingenieros capaces de justificar cada optimización.

Tipos de profiling: Lentes para ver tu programa ...

Tipos de profiling

CPU profiling muestra dónde gasta tiempo, memory profiling dónde gasta espacio, I/O dónde espera datos, GPU dónde paraleliza y network dónde pierde latencia.



Analogía médica

Es como tener rayos X, resonancia y tomografía para diagnosticar problemas. Netflix usa los cinco tipos para optimizar su streaming engine.



Objetivo práctico

Cada lente revela un tipo de cuello de botella y guía el tipo de optimización que realmente impactará la experiencia del usuario.



Herramientas: Del desktop al cloud

...

01

Ecosistema actual

Ecosistema 2024 incluye VTune, Perf, Instruments, cProfile, JProfiler, Visual Studio Profiler, Google Cloud Profiler y AWS X-Ray. Están disponibles, muchas gratis.

02

VTune

Intel VTune analiza CPU, GPU y FPGA; Google Cloud Profiler lleva el profiling al cloud; Perf integrado en kernel Linux con overhead casi cero.

03

Accesibilidad

No hay excusa para no medir; la barrera ya no es el acceso, sino la decisión de ser ingenieros de performance.

04

Beneficios prácticos

Estas herramientas permiten diagnosticar problemas de rendimiento con precisión, identificar cuellos de botella y aplicar optimizaciones efectivas.

Intel VTune: El Ferrari de los profilers



Características de VTune

VTune proporciona análisis comprehensivo para aplicaciones serial y multithreaded en CPU, GPU y FPGA, soportando C, C++, Python, Java y Fortran. Es la herramienta que usan Intel, Microsoft y Adobe internamente. Si aprenden VTune, pueden trabajar en cualquier empresa seria. Sus análisis de hotspots, threading, memoria y microarquitectura permiten ver exactamente dónde gasta tiempo el programa línea por línea, guiando optimizaciones con precisión quirúrgica.



AI Screenshot VTune: GPS del tráfico en tu código ...

Interfaz de VTune

La captura muestra lista de funciones ordenadas por tiempo CPU, gráfico de barras con porcentajes, código fuente anotado línea por línea y call stack con assembly.

Utilidad práctica

La columna Self Time indica dónde optimizar primero. Es como tener un GPS que muestra exactamente dónde está el tráfico en el código. Esta visión detallada convierte la optimización de adivinanza en ciencia, permitiendo justificar cada cambio con datos duros de ejecución real.



Linux Perf: Poder nativo en la terminal

01

Descripción de Perf

Perf es la herramienta nativa de Linux desarrollada por los creadores del kernel. Es menos amigable que VTune pero increíblemente poderosa y con overhead casi cero.

02

Uso en producción

Google la usa para perfilar servidores en producción. Los comandos `perf record` y `perf report` generan listas de funciones con porcentajes de samples y drill-down a assembly.

03

Importancia

Para sistemas de alto rendimiento, dominar perf es no negociable; es la forma más directa de entender qué hace el kernel con su código.

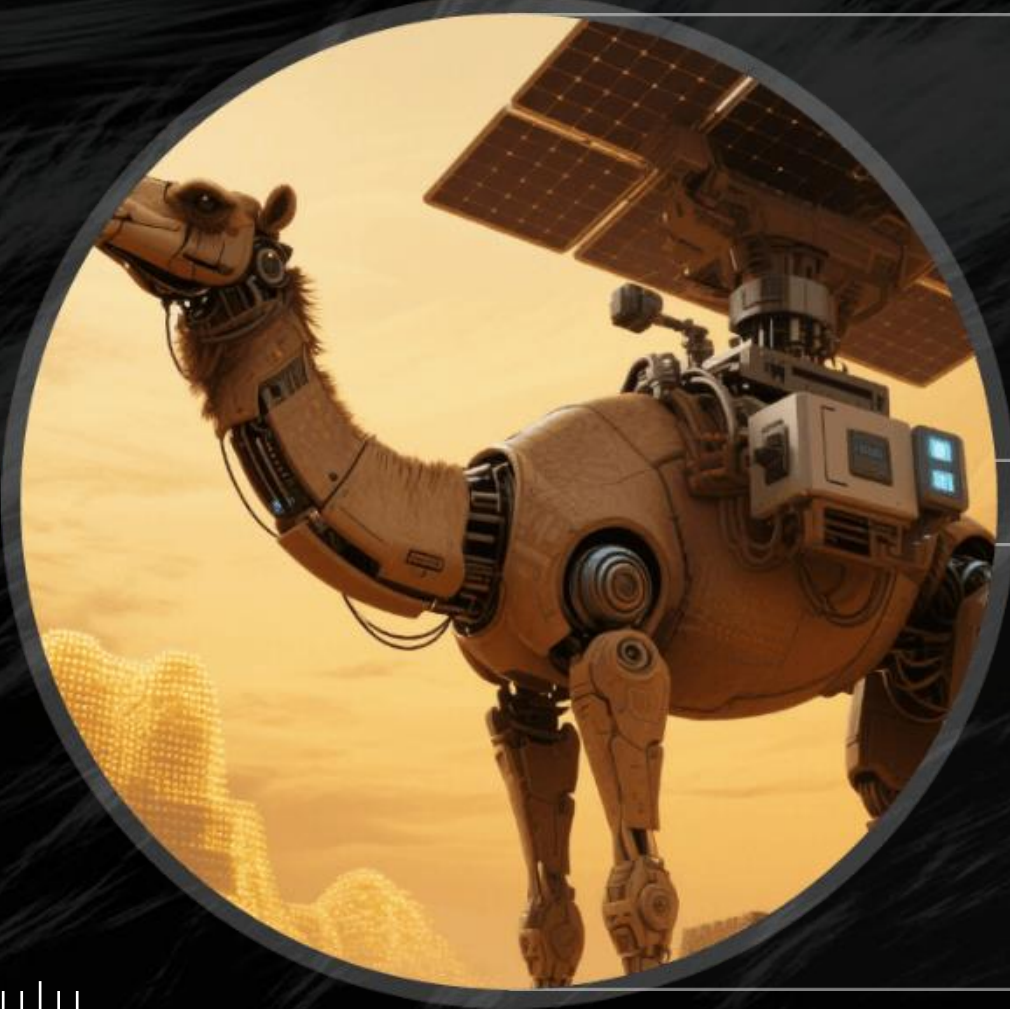
Screenshot Perf: Belleza en la simplicidad

Salida de Perf

La salida de perf report lista funciones con porcentajes del tiempo total de samples. Si una función muestra 45%, esa es la target de optimización número uno. Los números de la izquierda son porcentajes del tiempo total de samples. Esta simplicidad es poderosa: identifica rápidamente el cuello de botella sin distracciones. Perf convierte la terminal en un laboratorio de performance, permitiendo diagnosis en producción sin afectar a los usuarios.



Python cProfile: El profiler que ya tienes



Descripción de cProfile

cProfile viene built-in con Python y es sorprendentemente poderoso. Instagram lo usa para optimizar su backend Python. No necesitan instalar nada; ya está ahí. Es perfecto para proyectos académicos y profesionales iniciales.

Uso práctico

Con un simple `python -m cProfile` obtienen `ncalls`, `tottime`, `cumtime` por función. Es la puerta de entrada al profiling para desarrolladores Python que quieren pasar de funcionar a volar.

Screenshot cProfile: Oro puro para optimización



01

Salida de cProfile

La salida muestra tabla con ncalls, tottime, percall, cumtime y filename:lineno(function). Funciones ordenadas por tiempo acumulativo identifican bottlenecks claros.

Identificación de problemas

Si ven una función llamada un millón de veces con tiempo alto, encontraron el problema. cProfile convierte la intuición de lentitud en evidencia numérica, permitiendo priorizar optimizaciones y medir su impacto con precisión de milisegundos.

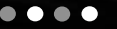
02

03

Beneficios

Esta herramienta es esencial para desarrolladores Python que buscan mejorar el rendimiento de sus aplicaciones, proporcionando datos concretos para tomar decisiones informadas.

Valgrind: Microscopio para tu programa



Características de Valgrind

Valgrind es el detective de memoria por excelencia en C++. Memcheck detecta memory leaks, Cachegrind simula caché y branch prediction, Callgrind perfila llamadas a funciones. Es más lento porque simula ejecución, pero la información es increíblemente detallada. Firefox y Chrome usan Valgrind para encontrar leaks. Es trabajo detectivesco fascinante que convierte programadores en ingenieros capaces de garantizar robustez y eficiencia.



Screenshot Callgrind: Mapa de ejecución visual



Visualización de Callgrind

KCacheGrind visualiza datos de Callgrind en call graph hermoso mostrando relaciones entre funciones, código fuente anotado con costo, estadísticas de caché hits/misses y timeline de ejecución.



Utilidad

Esta visualización convierte la complejidad de millones de llamadas en un mapa navegable, permitiendo entender el flujo real del programa y optimizar la arquitectura, no solo líneas aisladas.

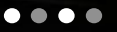


04

Metodología y Casos



Metodología científica de optimización



Pasos de la metodología

Baseline mide rendimiento inicial, Profile identifica hotspots con regla 80/20, Analyze entiende causas, Optimize modifica crítico, Verify confirma mejora sin regresiones y Repeat itera hasta objetivos.

Importancia de la baseline

Sin baseline no saben si mejoraron. Es el punto de partida para medir el impacto de las optimizaciones y garantizar que los cambios son positivos.

Necesidad de profiling

Sin profiling optimizan a ciegas. Identificar hotspots es esencial para enfocar esfuerzos en áreas críticas y no desperdiciar tiempo en código no relevante.

Verificación de resultados

Sin verify podrían introducir bugs. Confirmar mejoras sin regresiones es crucial para mantener la estabilidad y calidad del software.

Regla 80/20: Enfócate en el 20% crítico

...

01

Principio de Pareto

Principio de Pareto en programación es ley inmutable: 80% del tiempo se gasta en 20% del código. He visto estudiantes optimizar funciones que representan 1% del tiempo ignorando la que consume 70%.

02

Ejemplo práctico

Spotify tiene millones de líneas pero las optimizaciones críticas están en menos de 500. Medir primero, optimizar después es la diferencia entre ingeniería y artesanía.

03

Enfoque correcto

Enfocarse en el 20% crítico permite maximizar el impacto de las optimizaciones y evitar el desperdicio de recursos en áreas no relevantes.

Interpretando resultados: Vocabulario profesional



Definiciones clave

Self Time es tiempo exclusivo de la función, Total Time incluye llamadas, Call Count es número de invocaciones, Average Time es promedio por llamada y %Total es porcentaje del tiempo del programa.

Importancia en entrevistas

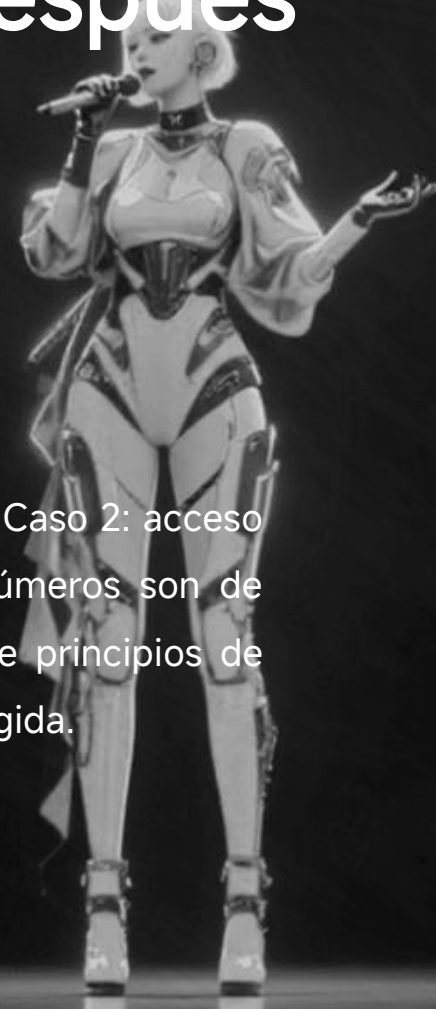
En entrevistas técnicas de Google, mencionar self time vs total time demuestra que entienden profiling. La diferencia determina si el problema está en la función o en sus dependencias, guiando el tipo de optimización necesaria.



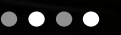
Casos de estudio reales: Antes y después

Ejemplos de mejora

Caso 1: búsqueda lineal $O(n)$ pasó de 2.5 s a 0.001 s con HashMap $O(1)$, mejora 2500×. Caso 2: acceso random a matriz pasó de 15 s a 0.8 s con acceso secuencial, mejora 19×. Estos números son de proyectos reales. No requieren algoritmos revolucionarios, solo aplicación correcta de principios de performance. La diferencia entre teórico y real se cierra con profiling y optimización dirigida.



Benchmark vs Profiling: Propósitos distintos



Diferencia entre ambos

Benchmark mide rendimiento general como 0-100 km/h en 10 s; profiling abre el capó y ve exactamente qué componente limita. Ambos son necesarios pero sirven propósitos diferentes.

Aplicación en la industria

Benchmark identifica problemas, profiling los localiza. Usar ambos es la diferencia entre saber que algo está lento versus saber exactamente qué línea arreglar. Es la combinación que usan los ingenieros de performance profesionales.

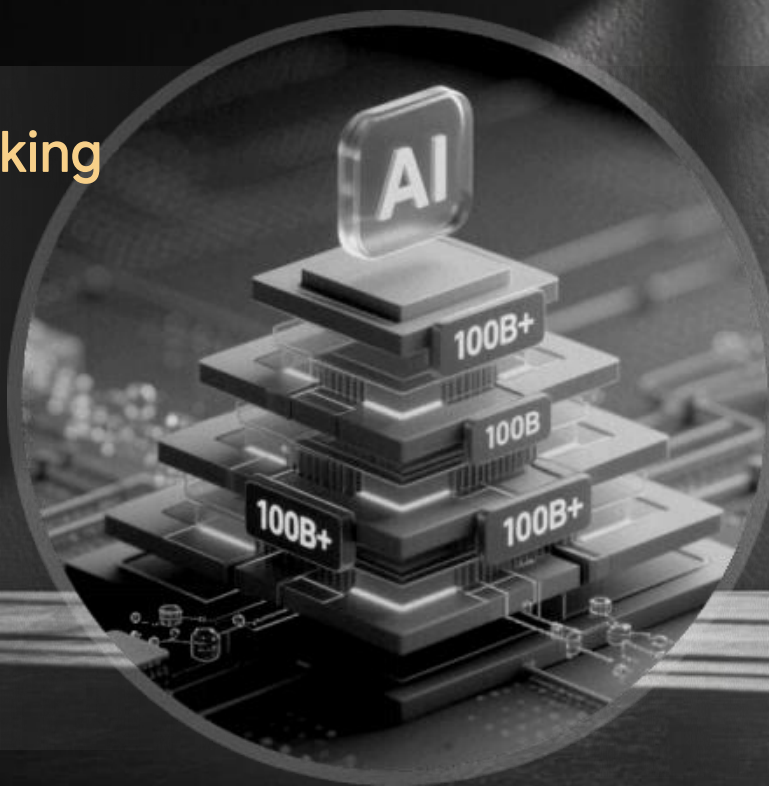
Importancia de la combinación

La combinación de benchmark y profiling permite una optimización completa, identificando problemas generales y luego localizando y resolviendo los específicos.

Microbenchmarking: Precisión para funciones

Herramientas de microbenchmarking

Herramientas para medir performance de código pequeño con precisión científica: Python timeit y pytest-benchmark, Java JMH, C++ Google Benchmark y Catch2, general hyperfine y time. Google usa Google Benchmark internamente para todas sus bibliotecas C++; JMH es estándar para Java.



Beneficios prácticos

Son esenciales cuando optimizan funciones individuales o algoritmos específicos, permitiendo decisiones basadas en microsegundos medidos con estadística rigurosa.

05

Laboratorio y Aplicación



Lab setup: Instala las herramientas reales



01

Herramientas a instalar

Van a instalar las mismas herramientas que usan Microsoft, Intel y NVIDIA: perf, valgrind, cProfile. No se preocupen por la complejidad inicial; están diseñadas para ser poderosas, no necesariamente fáciles.

02

Analogía con microscopio

Es como aprender a usar microscopio profesional. La preparación incluye descargar códigos de ejemplo, configurar scripts de medición y preparar datasets de prueba de 1K, 10K y 100K elementos para experimentos controlados.

03

Importancia de la preparación

La preparación adecuada es crucial para llevar a cabo experimentos de profiling exitosos y obtener resultados confiables y reproducibles.



Ejercicio 1: Profiling búsqueda lineal vs binaria ...

Objetivo del ejercicio

Comparar búsqueda lineal y binaria usando cProfile. Implementar ambos algoritmos, generar datasets ordenados de diferentes tamaños, medir con cProfile, analizar hotspots y call counts, graficar resultados vs predicción teórica.

Beneficios prácticos

Van a ver exactamente dónde y por qué búsqueda binaria es superior línea por línea, convirtiendo la intuición de $O(\log n)$ en evidencia medida de milisegundos y llamadas.

Ejercicio 2: Optimiza multiplicación de matrices



Pasos del ejercicio

Profilear implementación naive $O(n^3)$, identificar caché misses con Valgrind, implementar versión caché-friendly y medir mejora.

Resultados esperados

Verán mejoras de 5-10× simplemente cambiando orden de loops, convirtiendo teoría de localidad en aceleración real medible.

Problema de locality

Matrix multiplication es problema perfecto para demostrar optimización de caché: implementación naive tiene terrible locality; optimizada reorganiza loops para maximizar caché hits.

Importancia de la optimización

Este ejercicio demuestra cómo pequeños cambios en el código pueden tener un impacto significativo en el rendimiento, gracias a una mejor comprensión de la caché y la localidad de datos.



Ejercicio 3: Memory profiling y leaks

Descripción del ejercicio

Detectar y optimizar uso de memoria con Valgrind memcheck y memory_profiler de Python. Encontrar memory leaks en código C++ que habrían causado crashes en producción, optimizar estructuras Python que consumen memoria excesiva, medir memory footprint vs tamaño de datos y proponer mejoras de eficiencia espacial. Es trabajo detectivesco fascinante que convierte programadores en ingenieros capaces de garantizar robustez y escalabilidad.

Conexión TP1: Análisis empírico con profiling

...

Objetivo del TP1

TP1 adquiere dimensión adicional: no solo medirán tiempos, explicarán por qué difieren de predicciones teóricas usando profiling para investigar discrepancias.

Requisitos del TP1

Requisitos incluyen implementar algoritmos de diferentes complejidades, medir tiempos reales con datasets crecientes, comparar con predicciones y usar herramientas para identificar por qué la realidad se desvía del modelo matemático.

Beneficios prácticos

Este enfoque permite un análisis más profundo y una comprensión más completa del rendimiento real de los algoritmos en entornos prácticos.

“ ...

Conexión TP3: Optimización avanzada cuantificada

Objetivo del TP3

TP3 demuestra maestría: tomar código ineficiente, diagnosticar con profiling, aplicar optimizaciones específicas y documentar mejoras cuantificables.

Resultado esperado

Enfoque incluye identificar cuellos de botella, aplicar técnicas específicas, medir mejoras cuantitativas y documentar proceso. Resultado esperado: 25-50% mejora en rendimiento promedio demostrada con datos antes/después.

06

Errores, Futuro y Cierre



Errores comunes que invalidan resultados

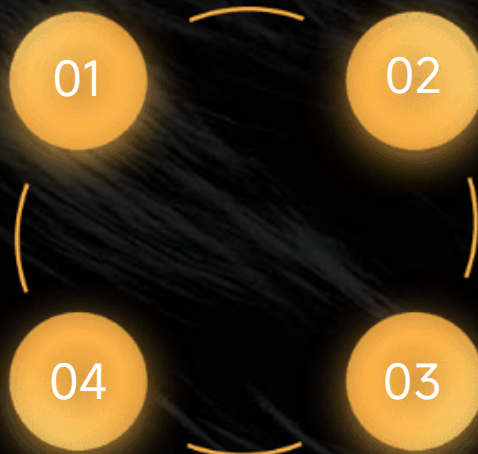
...

Errores en profiling

Profilear en modo debug es medir velocidad con frenos puestos; usar datasets pequeños es probar medicina con una persona; no considerar warm-up es ignorar estabilidad del sistema; optimizar código no crítico primero es desperdiciar tiempo.

Ejemplo práctico

Por ejemplo, no verificar la correccionalidad después de optimizar puede introducir bugs que afecten el rendimiento y la estabilidad del sistema.



Consecuencias

Estos errores causan que incluso ingenieros experimentados obtengan resultados inválidos, lo que puede llevar a decisiones incorrectas en la optimización.

Importancia de evitarlos

Conocer y evitar estos errores es crucial para obtener resultados confiables y aplicar optimizaciones efectivas.

Best practices: Controles de laboratorio

...

Prácticas recomendadas

Compilar con optimizaciones habilitadas refleja entorno real; usar datos representativos del uso real garantiza validez; ejecutar múltiples iteraciones elimina varianza estadística.



Aislamiento de procesos

Aislar procesos que interfieren elimina ruido y permite obtener mediciones precisas y confiables.

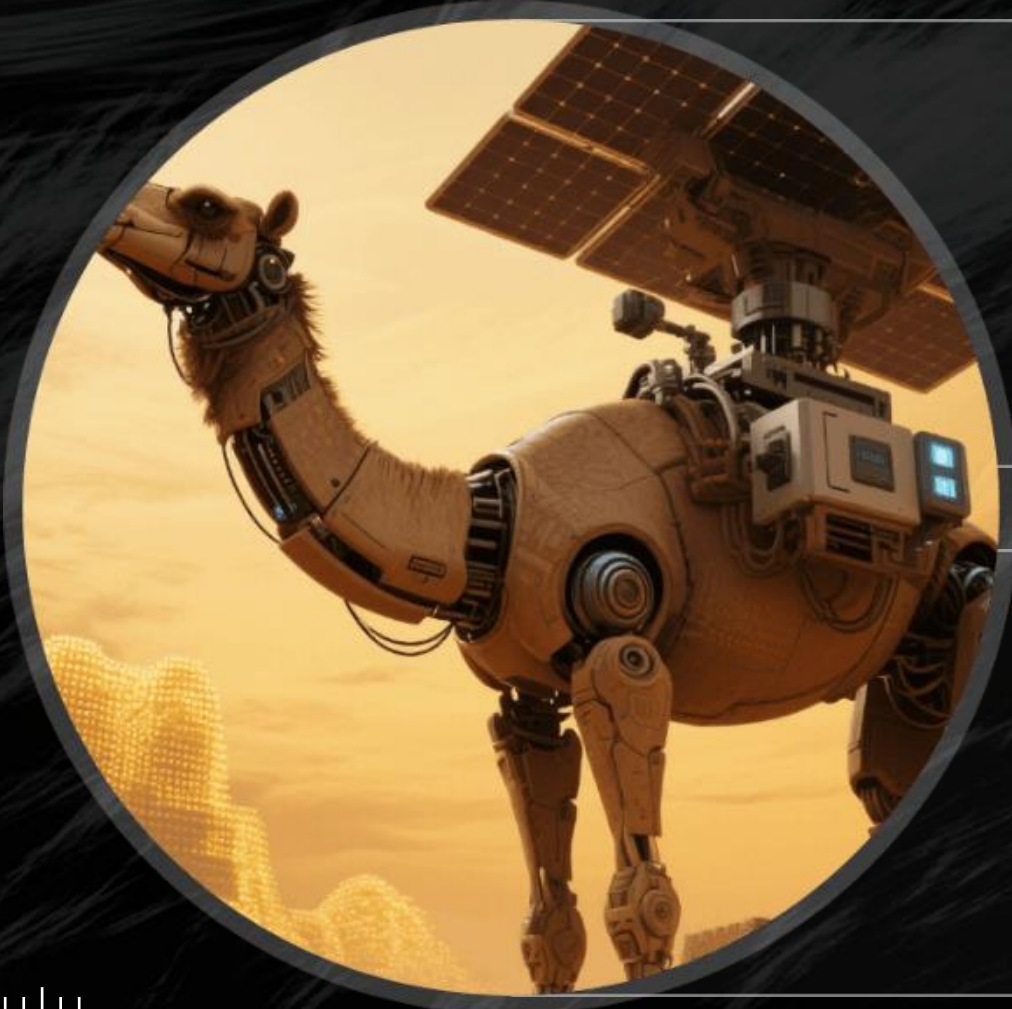


Documentación

Documentar cambios y mediciones permite reproducibilidad y facilita la comprensión de los resultados obtenidos.



Profiling distribuido: Más allá de una máquina



Descripción

Cuando trabajan en Uber o Airbnb sus sistemas corren en cientos de servidores. Profiling distribuido requiere coordinar mediciones entre nodos, correlacionar latencias de red y encontrar bottlenecks que cruzan servicios.

Herramientas y aplicaciones

Herramientas como Jaeger permiten seguir una request desde el celular del usuario hasta 15 microservicios. Es otro nivel de complejidad que requiere trazado distribuido y comprensión de arquitecturas modernas.

Actividad grupal: Competencia de profiling

Descripción de la actividad

Equipos de 3-4 personas reciben código base lento con múltiples ineficiencias. Tienen 45 minutos para identificar y optimizar usando herramientas disponibles. Gana el equipo con mayor mejora de rendimiento verificada.

Simula presión real: cuando Netflix se vuelve lento tienen este tiempo para diagnosticar y solucionar antes de que millones se quejen. Las herramientas deben ser segunda naturaleza, no algo que consultan en tutoriales.

Tendencias futuras: ML y profiling en tiempo real

....

Integración de IA

Herramientas están integrando IA para sugerir optimizaciones automáticamente. Intel ya desarrolla profilers que usan ML para recomendar cambios específicos.

Importancia

El futuro es profiling asistido por máquinas pero fundado en principios que hoy aprenden, lo que permitirá una optimización más eficiente y rápida.

Profiling en tiempo real

En 5 años las herramientas dirán no solo dónde está el problema sino exactamente cómo solucionarlo. Real-time profiling en producción se volverá estándar y energy profiling optimizará eficiencia energética.

Recursos adicionales: Profundiza tu maestría



Recomendaciones

Brendan Gregg es el gurú mundial de performance engineering en Netflix; su libro *Systems Performance* es la referencia mayor. [Quick-bench.com](https://quick-bench.com) permite comparar código C++ en el browser.

Curso avanzado

MIT 6.172 Performance Engineering es el curso más avanzado y está disponible online gratis. Estos recursos convierten conocimiento académico en maestría práctica.



Tarea: Encuentra casos donde profiling salvó proyectos

Descripción de la tarea

Investiguen 2 casos reales donde profiling salvó proyectos. Estos casos demuestran valor comercial real de estas habilidades. Stack Overflow es rápido no por accidente sino por profiling obsesivo. Quiero que encuentren historias de empresas que evitaron gastos millonarios o salvaron productos enteros gracias a diagnosis de performance. Presenten brevemente en próxima clase: problema, herramienta usada, optimización aplicada y resultado cuantificado.

Próximias actividades: Checkpoint y evaluación ...

Próxima clase

Próxima clase: técnicas de diseño de algoritmos avanzadas. TP1
checkpoint: vengan con dudas específicas, resultados parciales y problemas de implementación.

Importancia del checkpoint

No esperen al último día para descubrir que sus mediciones no tienen sentido. Lab 2 será profiling avanzado y optimización de memoria. Evaluación parcial preparará unidades 1-3.

Construcción hacia evaluaciones complejas

Estamos construyendo hacia evaluaciones más complejas que requieren dominio tanto teórico como práctico.



Síntesis: Optimización con medición es ciencia ...

Interpretación correcta de Knuth

Knuth dijo premature optimization is the root of all evil no porque optimizar sea malo sino porque hacerlo sin medición es mala. Con profiling la optimización nunca es prematura; es científica, dirigida y efectiva.

Importancia de la medición

Es la diferencia entre cirugía con bisturí versus hacha. Hoy aprendieron a convertirse en ingenieros de performance capaces de cruzar el puente entre la teoría hermosa y la realidad cruel, creando software que no solo funciona sino que vuela.

¡Gracias!

Mg. Marcelo F. Roldán

Computación II

